

# Delta Lake Fundamentals

## Executive Overview Matrix

| Feature / Dimension  | Plain Parquet                                 | Delta Lake                                                                                   |
|----------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------|
| Storage Format       | Open columnar file format (.parquet)          | Open storage layer built on Parquet + JSON transaction log                                   |
| ACID Transactions    | ✗ No (Partial writes can corrupt datasets)    | ✓ Yes (Full ACID compliance via <code>_delta_log</code> )                                    |
| Concurrency Control  | ✗ File-level locks or non-concurrent          | ✓ Optimistic Concurrency Control (OCC) / MVCC                                                |
| Time Travel / Audit  | ✗ Requires manual folder backup/versioning    | ✓ Built-in point-in-time reads by Version or Timestamp                                       |
| Schema Enforcement   | ✗ Schema drift silent or crashes readers      | ✓ Strict schema validation on write                                                          |
| Schema Evolution     | ✗ Manual table re-writes required             | ✓ Automatic merge ( <code>mergeSchema</code> ) or overwrite ( <code>overwriteSchema</code> ) |
| Mutations (DML)      | ✗ Re-write entire directories manually        | ✓ Native UPDATE, DELETE, and MERGE INTO (Upserts)                                            |
| Metadata Performance | ✗ $O(n)$ slow file listing over object stores | ✓ $O(1)$ fast metadata lookup using transaction log & checkpoints                            |

## 1. Delta Lake vs. Parquet

### Understanding Plain Parquet Limitations

Apache Parquet is an efficient, columnar binary file format optimized for read-heavy analytical workloads. However, when used directly as a storage layer in big data lakes, raw Parquet

presents severe operational challenges:

1. **No ACID Guarantees:** If a job writing 100 Parquet files crashes at file 50, the dataset is left in an incomplete, corrupted state. Readers cannot tell which files belong to completed transactions.
2. **Lack of Mutability:** Standard Parquet files are immutable. Performing an UPDATE or DELETE requires reading the entire partition/table into memory, filtering/modifying records, and re-writing all physical files.
3. **Expensive Object Storage Directory Listings:** Querying standard Parquet datasets requires scanning file systems to list every file path. As datasets grow to millions of files, directory listing latency becomes an  $O(n)$  performance bottleneck.
4. **Data Corruption via Concurrent Writes:** If two batch or streaming jobs attempt to write to the same Parquet directory simultaneously, files can be overwritten or interleaved without validation.

## How Delta Lake Extends Parquet

Delta Lake is an **open-source storage layer** sitting between Apache Spark (or other processing engines) and cloud object storage (such as ADLS Gen2, S3, or GCS).

- Delta Lake stores actual data records in **standard, compressed Parquet files**.
- It adds an ACID-compliant **Transaction Log (`_delta_log/`)** directory alongside the Parquet files.
- Readers process only the files explicitly marked as active in the transaction log, ignoring uncommitted or tombstoned files.

## 2. Delta Tables & Internal Storage Mechanics

## Anatomy of a Delta Table

When you create a Delta table, the underlying directory structure contains two components:

1. **Commit Files (.json):** Every operation that modifies a Delta table generates an ordered JSON commit file inside `_delta_log/`. Commit sequence numbers start at `00000000000000000000.json`.
2. **Checkpoint Files (.checkpoint.parquet):** Every 10 commits, Delta Lake automatically generates a Parquet checkpoint file that consolidates table state up to that point. This allows query engines to quickly rebuild the current table state without reading thousands of small JSON logs.

## Managed vs. External Delta Tables

| Dimension | Managed Table | External Table |
|-----------|---------------|----------------|
|-----------|---------------|----------------|

|                            |                                                                                 |                                                                                 |
|----------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Data Path Control</b>   | Spark/Metastore controls the physical path.                                     | User explicitly defines path via LOCATION 's3://...'                            |
| <b>DROP TABLE Behavior</b> | Deletes <b>both</b> catalog metadata <b>and</b> physical underlying data files. | Deletes <b>only</b> catalog metadata records. Data files remain intact on disk. |
| <b>Primary Use Case</b>    | Internal data mart tables maintained fully within workspace.                    | Enterprise lakehouse layers where storage lifecycle is decoupled from compute.  |

### 3. ACID Transactions in Delta Lake

Delta Lake brings relational database **ACID** (Atomicity, Consistency, Isolation, Durability) guarantees to cloud data lakes.

#### How Delta Achieves ACID Guarantees

- **Atomicity:** Any write to a Delta table either completes fully or fails completely. If a transaction fails mid-write, the data files written so far are never recorded in `_delta_log/`. Readers will never see partial writes.
- **Consistency:** Delta Lake checks schema compatibility on every write. If incoming records violate table constraints or column data types, the transaction fails before committing.
- **Isolation (Optimistic Concurrency Control - OCC):** Delta Lake uses OCC to manage concurrent reads and writes:
  1. **Record Read State:** The writer reads the current table version (e.g., Version 5).
  2. **Prepare Changes:** The writer writes new/modified Parquet files to storage.
  3. **Attempt Commit:** The writer attempts to write commit file `000006.json`.
  4. **Conflict Checking:** If another writer committed Version 6 first, Delta checks whether the concurrent modification overlaps with this writer's data.
    - If **no conflict** exists (e.g., updates to different partitions), Delta retries step 3 under Version 7 automatically.
    - If a **conflict** exists (e.g., concurrent updates to the same row/file), the transaction throws a concurrent modification exception.
- **Durability:** Once a commit file is written to `_delta_log/`, the transaction state is permanently saved in persistent storage.

### 4. Time Travel & Data Versioning

Because Delta Lake modifies tables by creating new Parquet files and recording changes in `_delta_log/` (rather than editing existing files in place), previous file versions remain available in

storage.

## Querying Historical Snapshots

You can query historical states of a Delta table using either **Version Number** or **Timestamp**.

### SQL Example

```
-- Query snapshot by explicit Version
SELECT * FROM gold_orders VERSION AS OF 12;

-- Query snapshot by Timestamp
SELECT * FROM gold_orders TIMESTAMP AS OF '2026-07-28 10:00:00';
```

### PySpark Example

```
# Query by Version
df_version_5 = (spark.read
    .format("delta")
    .option("versionAsOf", 5)
    .load("/mnt/datalake/gold/orders"))

# Query by Timestamp
df_historical = (spark.read
    .format("delta")
    .option("timestampAsOf", "2026-07-28 10:00:00")
    .load("/mnt/datalake/gold/orders"))
```

## Table Restoration (RESTORE)

If an erroneous pipeline run updates or deletes valid records, you can roll back the table instantly to a known good version:

```
RESTORE TABLE gold_orders TO VERSION AS OF 10;
```

## Maintenance and Cleanup (VACUUM)

Leaving old files indefinitely increases storage costs. The VACUUM command permanently deletes unreferenced Parquet files older than a specified retention threshold.

```
-- Default cleanup (removes unreferenced files older than 7 days / 168 hours)
VACUUM gold_orders;
```

-- Preview files eligible for deletion without physically removing them  
VACUUM gold\_orders RETAIN 168 HOURS DRY RUN;

– **Retention Warning:** Overriding safety controls (SET spark.databricks.delta.vacuum.parallelDelete.enabled = true) to run VACUUM ... RETAIN 0 HOURS while active batch/stream queries are reading the table will cause reader failures with FileNotFoundException.

## 5. Schema Evolution & Schema Enforcement

Delta Lake protects datasets from corrupted records while providing mechanisms to handle intentional payload changes.

### 1. Schema Enforcement (Schema Validation)

By default, Delta Lake blocks writes that do not match the destination table's schema.

- Extra columns in incoming data are rejected.
- Incompatible data type mismatches (e.g., writing a STRING into an INT column) are rejected.

### 2. Schema Evolution (mergeSchema)

When source systems add new fields, you can explicitly instruct Delta Lake to adapt table metadata to accommodate new columns during append/merge writes.

#### PySpark Example

```
(df_new_orders.write
  .format("delta")
  .mode("append")
  .option("mergeSchema", "true")
  .save("/mnt/datalake/gold/orders"))
```

#### Overwriting Schema (overwriteSchema)

If you intend to replace the entire table structure and data simultaneously:

```
(df_new_schema.write
  .format("delta")
  .mode("overwrite")
  .option("overwriteSchema", "true")
  .save("/mnt/datalake/gold/orders"))
```

## 6. Practical PySpark & SQL Code Reference

### 1. Table Creation & Ingestion

```
-- Create a Delta Table via SQL
CREATE TABLE gold.customer_sales (
  customer_id STRING,
  total_amount DOUBLE,
  last_order_timestamp TIMESTAMP,
  order_date DATE
)
USING DELTA
PARTITIONED BY (order_date);
```

### 2. DML Operations (UPDATE, DELETE, MERGE)

#### Delete Example

```
DELETE FROM gold.customer_sales
WHERE last_order_timestamp < '2024-01-01 00:00:00';
```

#### Update Example

```
UPDATE gold.customer_sales
SET total_amount = total_amount * 0.9
WHERE total_amount > 10000;
```

#### Upsert / Merge Example (MERGE INTO)

```
MERGE INTO gold.customer_sales AS target
USING silver.staged_updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
  UPDATE SET
    target.total_amount = source.total_amount,
    target.last_order_timestamp = source.last_order_timestamp
WHEN NOT MATCHED THEN
  INSERT (customer_id, total_amount, last_order_timestamp, order_date)
  VALUES (source.customer_id, source.total_amount, source.last_order_timestamp,
    source.order_date);
```

### 3. File Optimization & Clustering

To eliminate small file issues and accelerate query speeds:

-- Compact small files into ~1 GB chunks

```
OPTIMIZE gold.customer_sales;
```

-- Compact and cluster data using Z-Ordering on high-cardinality search keys

```
OPTIMIZE gold.customer_sales
```

```
ZORDER BY (customer_id);
```

## Summary

1. **Transaction Log:** Every write generates a new .json commit in `_delta_log/`.
2. **ACID Compliance:** Optimistic Concurrency Control prevents dirty reads and conflicting writes.
3. **Time Travel:** Retains file history to allow `VERSION AS OF` and `TIMESTAMP AS OF` reads.
4. **Schema Control:** Enforces schema boundaries by default; expands gracefully via `.option("mergeSchema", "true")`.
5. **Maintenance:** Use `OPTIMIZE / ZORDER BY` for compaction and indexing, and `VACUUM` for storage cleanup.